The mathematical model of the Extended system

Joint distance/speed belief, inferred context, information-seeking and a receding-horizon decision



AI-generated concept illustration, shared with the Baseline system; not a photograph or an assembly drawing.

Same observable task as the Baseline system. Approach a cardboard wall and verify that the ultrasonic sensor face is within 21–29 cm of it, with a nominal target of 25 cm. The Extended system changes how the controller estimates state and decides, not the task or the hardware interface.

This companion derives the equations behind every change the Extended system (`EXT1`) makes over the Baseline system (`V1`), across four milestones: a two-mode joint distance/speed belief, a Gaussian information proxy, a two-step receding-horizon planner with an explicit no-progress cost, and a fair rule-based comparator (Milestone 1); a persistent per-context memory, the diagnosed reason it alone did not help, and the accumulated-evidence recognition mechanism that then made it work (Milestone 2, Sections 11–12); an explicit, costed sensor-bias reference that breaks the distance/bias identifiability problem (Milestone 3, Section 13); and a frozen evaluation on genuinely held-out seeds, plus a new outlier-reading robustness test (Milestone 4, Section 16); and a later, separate validation pass, the *Transfer system*, testing whether the learned context library transfers to a different goal under a persistent, deployment-specific speed bias (Section 17), positive preliminary evidence for faster regime recognition across tasks, reported together with the metrics it does not move, not folded into the four-milestone count above. It assumes familiarity with `baseline_v1/Mathematical_Model.pdf` for the shared sensor model and the physical pulse-

to-motor mapping, which are not re-derived here.

Start with: the Extended-system tutorial or the Extended-system README. Keep this PDF beside those files so relative links resolve.

Prepared by Florent Ouabo Kamkumo, PhD
3 October 2026 / Extended System Milestone 4 companion v4.0
Status: computer-tested desktop prototype; no physical validation. All six items of the project's improvement plan are now addressed (Section 19); physical hardware validation remains entirely separate and not yet attempted.

Contents

1 The task and what the Extended system adds **2**

2 Variables and the augmented belief **3**

3 Observation model **3**

4 Context-mixture movement model **4**

5 Bayesian update and context reweighting **5**

6 The information proxy **5**

7 The one-step planner this release replaces **5**

8 The receding-horizon planner **6**

8.1 Worked example: how the penalty changes a real trajectory 7

9 A fair rule-based comparator **7**

10 Verification, budgets and the completed factorial **8**

11 Context memory: hypothesis, a negative result, and a diagnosis **8**

11.1 A persistent per-context library 8

11.2 The negative result 9

11.3 Diagnosis: the bottleneck is recognition, not precision 9

12 Recognising a regime from accumulated evidence, then reusing it **9**

12.1 A one-sided CUSUM 10

12.2 The jump: discrete reuse 10

12.3 Threshold selection on development data 10

12.4 The result 11

13 Breaking the distance/bias confound: an explicit reference **11**

13.1 A known-distance reference, not a cleaner oracle 11

13.2 Propagating the estimate and its uncertainty 12

13.3 Results 12

14 Calibration **12**

15 Default parameters: a complete reproducible specification **13**

16 A frozen, held-out evaluation **13**

16.1 An added robustness scenario: outliers 14

16.2 Replication report 14

17 A cross-task transfer test: the Transfer system, and Objective O3 **14**

17.1 Task B: the same physics, a different goal 14

17.2 A persistent per-deployment speed bias 15

17.3 Three conditions on Task B 15

17.4 Results: still nothing on the original metrics 16

17.5 A sharper metric: post-switch reconvergence 16

17.6 Conclusion: positive preliminary evidence for O3, with its actual scope 17

18 Reading route and tutorial crosswalk **17**

19 Status: all six improvement-plan items addressed **18**

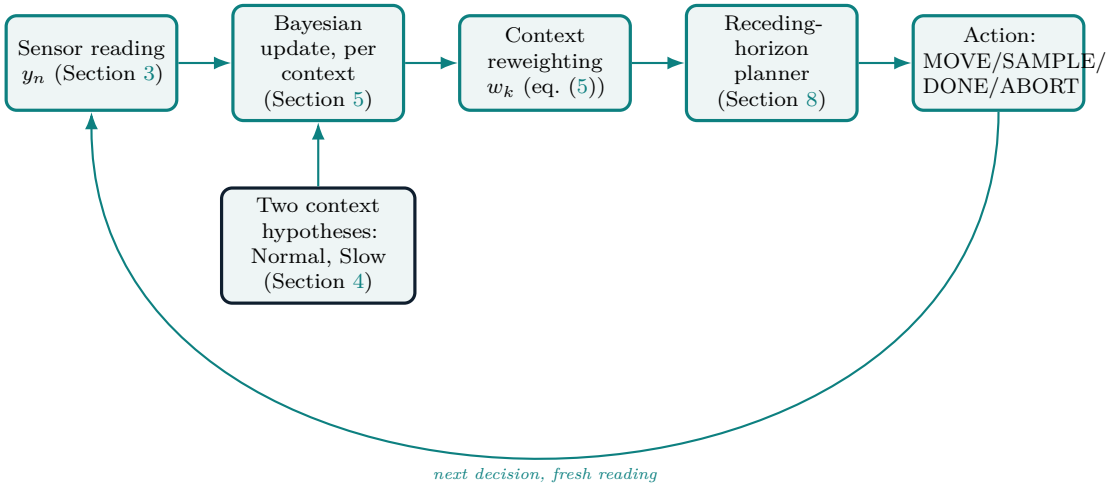
1 The task and what the Extended system adds

The Baseline system answers: *given an uncertain distance measurement and an imperfect movement model, should the robot move a little, measure again, or stop?* with a scalar Bayesian estimator and a one-step rule. The Extended system answers a richer question: *given uncertain distance **and** speed, under one of several possible movement contexts, what short sequence of actions over the next few decisions best balances progress, cost and the value of more information?*

Three extensions are layered on the shared observation model and verification rule (Sections 2–10 summarise what is shared; Sections 8–9 derive what is new):

1. A joint state (d, v), distance and effective speed, tracked as a weighted mixture of Gaussians, one per movement context (normal/slow), rather than a single scalar.
2. A Gaussian information proxy that can reward an action for the speed uncertainty it is expected to resolve, not only for reducing distance error.
3. **New this release.** A receding-horizon planner that scores short (two-decision) action sequences instead of one decision at a time, with an explicit penalty for sequences that make no progress, and a fair rule-based comparator (`FairRuleBaseline`) that gives the Baseline system's decision philosophy the same action menu as the Extended system, so that any remaining performance gap is attributable to the decision architecture rather than to a richer set of available actions.

The one-cycle shape of everything below. Before the equations, the loop every section after this one is a piece of:



One decision = one pass around this loop. Every section below derives one box or one arrow; the box/arrow it derives is named in each section's own heading.

Do this in the Extended-system tutorial: Section 1, The Extended system: what changes. See the browser tutorial or the PDF tutorial.

2 Variables and the augmented belief

Work in centimetres and seconds, as in the Baseline system. The augmented state per context $k \in \{1, \dots, K\}$ ($K = 2$ for {normal, slow}, or $K = 1$ for the context-free `adaptive` variants) is

$$x_k = (d_k, v_k)^\top, \qquad P_k \in \mathbb{R}^{2\times 2}_{\succeq 0}, \qquad w_k \in [0, 1],\ \textstyle\sum_k w_k = 1.$$

Example, in plain numbers, before the symbol table below. The robot starts roughly 70 cm from the wall and takes its first reading: 69.4 cm (noisy, true sensor sd $\approx$ 1.2 cm). With no speed evidence yet, the belief holds *two* running guesses side by side, weighted by the fixed starting split (0.8/0.2, Section 4): a "normal" guess $x_{\text{normal}} = (69.4, 8.0)$, that is, 69.4 cm, 8.0 cm/s, the calibrated default speed, with weight $w_{\text{normal}} = 0.8$; and a "slow" guess $x_{\text{slow}} = (69.4, 3.0)$ with weight $w_{\text{slow}} = 0.2$. After a move and a second reading, whichever guess's predicted distance matches what was actually observed starts winning weight (Section 5's reweighting rule); within a handful of readings one w_k is close to 1 and the other close to 0. Every d_k, v_k, w_k in the rest of this companion is exactly this: not an abstract symbol, but one of these two running, per-context guesses, updated one reading at a time.

Symbol	Meaning	Unit
d_k, v_k	Per-context mean distance and effective speed	cm, cm/s
P_k	Per-context 2×2 covariance of (d_k, v_k)	cm^2, cm^2/s, cm^2/s^2
w_k	Posterior probability of context k	dimensionless
$\bar{x}, \bar{P}$	Mixture (reported) mean and covariance, eq. (4)	as above
ρ	Speed mean-reversion rate per move (0.80 with contexts, 1 without)	dimensionless
Q_d, Q_v	Added distance/speed process variance per move	cm^2, cm^2/s^2
T	Context transition matrix, fixed at $\text{diag}(.96) + .04(1 - \text{diag})$	dimensionless
H	Planning horizon (2 decisions in this release)	dimensionless
$\Delta_{\min}$	No-progress displacement threshold (1 cm)	cm
λ	No-progress penalty magnitude (1.0, same units as the score)	dimensionless

This augments, rather than replaces, the Baseline system's scalar (μ, P): setting $K = 1$ and dropping v recovers a model of the same shape as the Baseline system's belief.

Shared with the Baseline system: see `baseline_v1/Mathematical_Model.pdf`, Section 2, Variables, units and the complete internal state.

3 Observation model

The Extended system keeps the Baseline system's bias-corrected linear-Gaussian observation model for distance,

$$y_n = d_n + (b - \hat{b}) + \nu_n, \qquad \nu_n \overset{\text{model}}{\sim} \mathcal{N}(0, R),$$

applied identically to every context's distance component; speed is never observed directly, only inferred through its effect on successive distance predictions. The innovation gate rejects an observation when $|y_n - \bar{d}^-| > 5\sqrt{\bar{P}^-_{dd} + R}$ (five standardised units; a widened version of the Baseline system's $\max\{8, 4\sqrt{P^- + R}\}$ cm rule, since the reference implementation (`cognitive/model.py`, `Belief.observe`) expresses the test directly in standardised, not raw, units). `FairRuleBaseline` (Section 9) instead reuses the Baseline system's exact raw-unit test, since it is a rule-based comparator, not an estimator of speed.

Shared with the Baseline system: see `baseline_v1/Mathematical_Model.pdf`, Section 3, From ultrasound time to a distance observation.

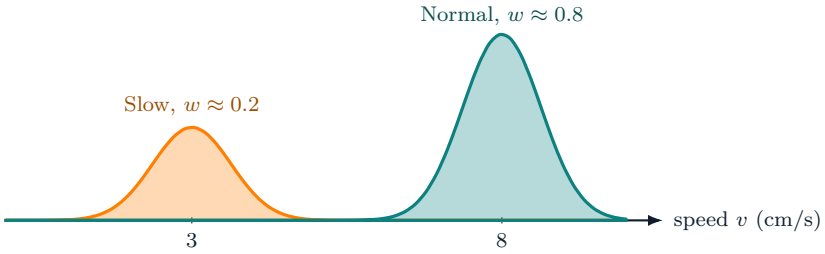
4 Context-mixture movement model

Conditional on context k being active and a pulse of duration τ,

$$d_{k,n+1} = d_{k,n} - v_{k,n}\tau - c + w_n^d, \qquad v_{k,n+1} = \rho\, v_{k,n} + (1-\rho)m_k + w_n^v, \tag{1}$$

where c is coast distance and m_k is context k's calibrated nominal speed (about 8 and 3 cm/s for normal and slow). Context itself follows a fixed Markov chain with transition matrix T ($T_{kk} = .96$, off-diagonal .04 for $K = 2$), applied once per *motor* decision, not once per stationary reading; repeated SAMPLE actions leave w_k unchanged by this mechanism (verified by `test_stationary_sampling_does_not_identify_speed`).

What "a weighted mixture of Gaussians, one per context" looks like. Section 2's opening phrase, drawn rather than only stated: two candidate speed beliefs held at once, each a bump of uncertainty around its own current mean, scaled here by the starting weights $w_{\text{normal}} = 0.8, w_{\text{slow}} = 0.2$ (sketch only, not to the filter's actual calibrated scale):



As readings arrive, eq. (5) shifts weight toward whichever bump better predicts them (narrowing toward one spike, Section 5); eq. (2)–(3) below is what moves and reshapes each bump between readings.

Moment-matching mixture collapse. Before propagating, the implementation (`Belief._advance`) first mixes each outgoing context j's prior moments over all incoming contexts i, weighted by $w_i T_{ij}/w_j^-$ where $w^- = wT$ is the predicted context prior:

$$\tilde{w}_{i\to j} = \frac{w_i T_{ij}}{w_j^-}, \qquad \tilde{x}_j = \sum_i \tilde{w}_{i\to j}\, x_i, \qquad \tilde{P}_j = \sum_i \tilde{w}_{i\to j}\Big(P_i + (x_i - \tilde{x}_j)(x_i - \tilde{x}_j)^\top\Big). \tag{2}$$

This is the interacting-multiple-model (IMM) mixing step: it lets a context's belief absorb information from other contexts in proportion to how likely a transition from each of them was, rather than keeping K fully independent filters. Equation (1) is then applied per context to $(\tilde{x}_j, \tilde{P}_j)$, giving the linear update

$$x_j' = F_\tau\, \tilde{x}_j + \begin{pmatrix} -c \\ (1-\rho)m_j \end{pmatrix}, \qquad P_j' = F_\tau\, \tilde{P}_j\, F_\tau^\top + \operatorname{diag}(Q_d, Q_v), \qquad F_\tau = \begin{pmatrix} 1 & -\tau \\ 0 & \rho \end{pmatrix}. \tag{3}$$

`Belief.predict` applies (2)–(3) in place; `Belief.predicted` (new this release) returns the same result *without* mutating the stored belief, and `Belief._advance` is the pure function both call; this is what lets the planner (Section 8) roll the belief forward hypothetically, more than one step, without disturbing the real filter. The reported (mixture) mean and covariance at any point are

$$\bar{x} = \sum_k w_k x_k, \qquad \bar{P} = \sum_k w_k (P_k + (x_k - \bar{x})(x_k - \bar{x})^\top). \tag{4}$$

Do this in the Extended-system tutorial: Section 3, Model and estimation. See the browser tutorial or the PDF tutorial.

5 Bayesian update and context reweighting

On an accepted reading y_n, each context's distance component is updated by an ordinary Kalman step with gain $g_k = P_k[:,0]/(P_{k,dd}+R)$, applied to the full 2-vector x_k (so a distance innovation also nudges the speed estimate through the off-diagonal of P_k, which is exactly how a single move followed by a reading lets the filter learn about speed from distance alone). The implementation applies the update in Joseph form for numerical robustness,

$$x_k \leftarrow x_k + g_k(y_n - d_k), \qquad P_k \leftarrow (I - g_k e_1^\top)P_k(I - g_k e_1^\top)^\top + g_k g_k^\top R, \qquad e_1 = (1,0)^\top.$$

Context weights update by their per-context marginal likelihood, exactly as a Gaussian-mixture filter's weights:

$$\ell_k = \log w_k - \tfrac{1}{2}\Big[\log(2\pi(P_{k,dd}+R)) + \frac{(y_n - d_k)^2}{P_{k,dd}+R}\Big], \qquad w_k \leftarrow \frac{e^{\ell_k - \max_j \ell_j}}{\sum_j e^{\ell_j - \max_j \ell_j}}. \tag{5}$$

The $\max_j \ell_j$ subtraction is a standard log-sum-exp stabiliser; it does not change the normalised result. A context that predicts the observed reading much better than the other accumulates weight geometrically fast; this is the mechanism behind `test_context_inference_and_covariance`, which checks that forty-five slow-context observations drive $w_{\text{slow}} > 0.8$.

Shared with the Baseline system: see `baseline_v1/Mathematical_Model.pdf`, Section 5, Deriving the scalar Bayesian update.

6 The information proxy

For a hypothetical pulse of duration τ (evaluated on the *current*, not mixture-mixed, state, via `Belief.information`), approximate the one-step-ahead joint covariance as $P^+ = F_\tau \bar{P} F_\tau^\top + \mathrm{diag}(Q_d, Q_v)\mathbf{1}_{\tau>0}$. The value of a subsequent distance reading for *speed* is captured by comparing the marginal speed variance before and after conditioning on a hypothetical distance observation:

$$P^+_{\text{post}} = \max\Big(10^{-9},\, P^+_{vv} - \frac{(P^+_{vd})^2}{P^+_{dd}+R}\Big), \qquad \mathrm{Info}(\tau) = \max\Big(0,\, \tfrac{1}{2}\log\frac{P^+_{vv}}{P^+_{\text{post}}}\Big).$$

This is the differential entropy reduction of a Gaussian under a linear-Gaussian observation, restricted to the speed marginal; it is a proxy for the value of information about speed specifically, not an expected free energy functional and not a measure of information about context (which would require comparing full mixture predictive distributions across hypotheses, not a single Gaussian's conditional variance). $\mathrm{Info}(0) = 0$ since $F_0 = I$ and no process noise is added, so SAMPLE never buys speed information in this proxy, consistent with speed only ever being learned through move-then-observe pairs (Section 4).

Do this in the Extended-system tutorial: Section 3, Model and estimation. See the browser tutorial or the PDF tutorial.

7 The one-step planner this release replaces

Through the previous Extended-system release, each decision scored the three candidate durations $\tau \in \{0, .12, .24\}$ once, using a closed-form one-step approximation

$$\widehat{P}(\tau) = \bar{P}_{dd} - 2\tau\bar{P}_{dv} + \tau^2\bar{P}_{vv} + Q_d\mathbf{1}_{\tau>0}, \qquad \mathrm{Score}_1(\tau) = \frac{(\bar{d} - v\tau - 25)^2 + \widehat{P}(\tau)}{400} + 0.08\,\kappa(\tau) - 0.8\,\mathrm{Info}(\tau),$$

with $\kappa(\tau) = \tau + .20$ for a move and $\kappa(0) = .15$ for SAMPLE, and τ excluded if $\bar{d} - v\tau - 2.33\sqrt{\widehat{P}(\tau)} - .5 < 21$. This is a one-step, closed-form approximation to the mixture dynamics (it operates on aggregate moments directly, not through (2)–(3)). Because it only ever compares *this instant's* alternatives, a state in which every SAMPLE-vs-MOVE trade-off narrowly favours SAMPLE (for instance because remaining distance uncertainty is already small and a move's added process variance Q_d outweighs its small expected progress) has no mechanism to recognise that committing to a plan which samples now and moves next is preferable to sampling indefinitely. With information-seeking switched off (Info $\equiv 0$ in the score), this could make the all-SAMPLE action a persistent local optimum. Section 8 replaces Score_1 with a two-step, forward-simulated score that both uses the real mixture dynamics and adds an explicit term for this specific failure mode.

8 The receding-horizon planner

At each decision, enumerate every pair $(\tau_1, \tau_2) \in \{0, .12, .24\}^2$ ($H = 2$, nine pairs). For each pair, roll the belief forward twice using (2)–(3) without mutating it (`Belief.predicted` then `Belief._advance`), obtaining mixture moments $(\bar{d}_1, \bar{P}_1)$ after the first step and $(\bar{d}_2, \bar{P}_2)$ after the second. Exclude a pair the moment either half-step would predict an unsafe overshoot,

$$\tau_i > 0 \text{ and } \bar{d}_i - 2.33\sqrt{\bar{P}_{i,dd}} - .5 < 21 \quad \Longrightarrow \quad \text{exclude } (\tau_1, \tau_2).$$

Among the surviving pairs, score

$$\text{Score}_2(\tau_1, \tau_2) = \underbrace{\frac{(\bar{d}_2 - 25)^2 + \bar{P}_{2,dd}}{400}}_{\text{end-of-horizon goal}} + \underbrace{0.08\,\kappa(\tau_1) + 0.08\,\kappa(\tau_2)}_{\text{accumulated cost}} - \underbrace{0.8\,\text{Info}(\tau_1)}_{\text{information, first step only}} . \tag{6}$$

Information is evaluated only for τ_1 (the action about to be executed), matching the principle that information-seeking should improve an already competent controller rather than substitute for one: the end-of-horizon goal term, not a speculative two-step information term, is what the no-progress correction below targets.

Explicit no-progress cost. Let $\Delta(\tau_1, \tau_2) = \bar{d} - \bar{d}_2$ be the pair's net predicted displacement from the current mean $\bar{d}$. If *some* surviving pair achieves $\Delta > \Delta_{\min} = 1$ cm, add a fixed penalty $\lambda = 1.0$ to the all-SAMPLE pair's score:

$$\text{Score}_2(0,0) \;\leftarrow\; \text{Score}_2(0,0) + \lambda \cdot \mathbf{1}\Big[\max_{(\tau_1,\tau_2)} \Delta(\tau_1, \tau_2) > \Delta_{\min}\Big]. \tag{7}$$

Execute $\tau_1^\star$ from $\arg\min \text{Score}_2$ (receding-horizon control: only the first action is taken; the next decision re-plans from a fresh observation). The penalty only ever fires when it is not vacuous, i.e. only when a safe, progress-making alternative actually exists, so it cannot force a move when every move is judged unsafe (in that case (7)'s indicator is false and the controller is left to SAMPLE, which is the correct conservative behaviour).

What this does and does not establish. Equation (7) is a heuristic mitigation for the specific failure identified in Section 7 (a persistent all-SAMPLE optimum when a two-step view and the current cost weights would otherwise make waiting look best), not a proof that no planner of this family can stall under any parameter setting, nor a general solution to exploration-exploitation trade-offs. $H = 2$ is also a specific, small choice; the implementation and this derivation extend directly to $H = 3$ by enumerating $\{0, .12, .24\}^3$ and chaining one more `_advance` call, at 3× the per-decision enumeration cost.

8.1 Worked example: how the penalty changes a real trajectory

The following is read off an actual run of `CognitiveController(CAL, contexts=True, informative=False)` started at true distance 70 cm with calibration `CAL` from `tests/test_cognitive.py` (seeded Gaussian sensor noise, no model misspecification), not an invented illustration.

At decision $t = 30$ (seconds elapsed ≈ 6.0), the belief has settled to $\bar{d} \approx 29.667$ cm, $\bar{P}_{dd} \approx 0.1149$ cm^2 ($\sigma_d \approx 0.339$ cm), just outside the tight verification band ($|29.667 - 25| + 1.64(0.339) = 5.22 > 4$). The nine candidate pairs' raw scores (before (7)) are, to four decimals,

τ_1	τ_2	Score$_2$ (no penalty)	Δ (cm)	safe?
0	0	0.0424	2.00	yes
0	.12	0.0457	3.00	yes
0	.24	0.0505	3.85	yes
.12	.12	0.0541	3.99	yes
.24	.24	0.0674 (worst of nine)	6.1	yes

(remaining pairs omitted; all nine are safe at this state). Without (7), $(0, 0)$ is narrowly the best plan (0.0424 against 0.0457 for the next best), exactly the kind of near-tie a one-step planner has no way to see past, and the seed from which repeated SAMPLE could persist. Because the best feasible displacement (≈ 3.99 cm for $(.12, .12)$, and others) exceeds $\Delta_{\min} = 1$ cm, (7) adds $\lambda = 1.0$ to $(0, 0)$'s score, making $(0, .12)$ (score 0.0457) the selected plan. The *executed* action is still SAMPLE ($\tau_1 = 0$), the controller takes one more reading, it is not forced to move this instant, but it no longer commits to a plan that samples forever.

One decision later ($t = 31$, after that extra reading has tightened the belief slightly), re-planning from the fresh state selects $\tau_1 = 0.24$ outright: the actual recorded action is `MOVE`, bringing the estimate to $\bar{d} \approx 29.528$ before the move and to about 27.6 cm after it. Verification succeeds three decisions later, at $t = 34$ ($\bar{d} \approx 27.1$ cm), ending the episode `DONE`. The full sequence, SAMPLE at $t = 28, 29, 30$, MOVE at $t = 31$, SAMPLE/SAMPLE/DONE at $t = 32$–34, is reproducible with `experiments/run_simulation.py`'s `adaptive_no_information` policy (seed 10000, scenario `normal`; see `results_ext1/trace_normal_adaptive_no_information.json`).

Do this in the Extended-system tutorial: Section 4, Decisions and control limits. See the browser tutorial or the PDF tutorial.

9 A fair rule-based comparator

The Baseline system's `mini` policy (`baseline_v1/firmware/belief_controller.py`) only ever offers $\tau = 0.12$ s, so no comparison against it can separate "a better decision rule" from "a wider action menu." `FairRuleBaseline` (`cognitive/baselines.py`) keeps the Baseline system's rule-based philosophy, the same verification rule (9) below, the same invalid/inconsistent-reading handling, the same progress-abort check, *no* joint speed estimation, but chooses the *longest* duration from the shared menu $\{.24, .12\}$ (checked in that order) that clears the same overshoot margin the Baseline system uses for its single pulse:

$$\tau^\star = \max\Big\{\tau \in \{.24, .12\} : \ \mu - 1.64\sqrt{P} - (\hat{v}\tau + \hat{c}) - .5 \geq 21\Big\} \cup \{0\}. \tag{8}$$

The Baseline system's original rule is the special case of (8) with menu $\{.12\}$ only: (8) does not change *how* the controller decides whether a step is safe, only how large a menu it is allowed to pick from, which is exactly the property needed to make "does the decision architecture matter, holding the action set fixed" a well-posed comparison. Verification is unchanged from the Baseline system:

$$|\mu - 25| + 1.64\sqrt{P} \leq 4, \tag{9}$$

with the same three-consecutive-pass rule and the same periodic progress check every eight moves. In the `normal` scenario (`results_ext1/trace_normal_v1_fair.json`), (8) selects $\tau = 0.24$ on every move, roughly halving the move count against the original single-pulse Baseline system for the same trial, the cleanest available evidence that part of the Baseline system's apparent disadvantage was its action menu, not its rule. The wider menu is not uniformly better, though: in the `slow` and `switch` scenarios `v1_fair` scores markedly *below* the original Baseline system (`results_ext1/summary.csv`), because (8) still assumes the fixed, uncalibrated-for-this-regime speed $\hat{v}$; committing to the longer pulse on a wrong speed overshoots by more than the short pulse would have. A wider action menu only helps when paired with a dynamics estimate that can tell when it is safe to use it.

Do this in the Extended-system tutorial: Section 4, Decisions and control limits. See the browser tutorial or the PDF tutorial.

10 Verification, budgets and the completed factorial

DONE/ABORT conditions, the 80 s / 100-move / 350-decision budgets, and the three-consecutive-pass verification rule are unchanged from the Baseline system's Mini policy (cross-referenced in Section 9) and shared by every Extended-system policy. What changed this release is which *comparators* share them: `adaptive_no_information` (`CognitiveController(contexts=False, informative=False)`) completes the 2×2 (context $\times$ information) factorial begun by `adaptive`, `ext1_no_information` and `ext1`, so each of the two architectural ingredients can be attributed independently, holding the planner, action menu and budgets fixed across all four cells.

Shared with the Baseline system: see `baseline_v1/Mathematical_Model.pdf`, Sections 6–7, Verification: what DONE means mathematically; The implemented Mini movement decision.

11 Context memory: hypothesis, a negative result, and a diagnosis

The remaining sections of this companion document a second milestone within the same release cycle, reached by a route worth recording precisely because it did not go directly from hypothesis to success. The question motivating it: once a regime (normal or slow) has been experienced, does recognising it again let the controller adapt faster than estimating its dynamics from scratch a second time, faster *reuse*, not merely accurate classification? Testing this requires a regime to be revisited within a single episode, which no earlier scenario provided; the `repeated` scenario (alternating normal/slow every 8 moves) was added for exactly this purpose.

11.1 A persistent per-context library

Equation (3) mean-reverts each context's speed toward m_j, a *fixed* calibration constant, through state $\tilde{x}_j$ that equation (2) continuously re-mixes across every context at every step, so nothing about a context's speed actually persists once belief mass moves away from it. `Belief` (`memory=True`) adds a separate pair of arrays, $(\text{lib_mean}_k, \text{lib_var}_k)$, that (2) never touches, updated only when context k is confidently active ($w_k > 0.7$) by an ordinary Kalman step treating the instantaneous per-component estimate as one noisy observation of the context's long-run speed:

$$g = \frac{\text{lib_var}_k}{\text{lib_var}_k + P_{k,vv}}, \qquad \text{lib_mean}_k \mathrel{+}= g\,(v_k - \text{lib_mean}_k), \qquad \text{lib_var}_k = \max(\text{floor}, (1-g)\,\text{lib_var}_k). \tag{10}$$

Equation (3)'s reversion target m_j is then replaced by lib_mean_j. Because the library is written only when confidently active and is immune to (2)'s mixing, a context's learned speed should, in principle,

survive untouched while that context is inactive: confirmed directly, seeding $\text{lib_mean}_{\text{slow}} = 2.0$ (away from the 3.0 calibration prior) and predicting forward 20 steps with no slow-context evidence leaves it at exactly 2.0, whereas the same experiment on the *un*protected instantaneous state $\text{states}_{\text{slow},v}$ reaches 2.886 after a single step and overshoots to 4.09 by step 10, actively corrupted by the other context's state through the mixing step, not merely decaying.

11.2 The negative result

Despite the library behaving exactly as designed, `ext1_memory` measured on `repeated` showed no reconvergence benefit over plain `ext1`; if anything, slightly worse and less reliable (the table in Section 12.4 below gives the exact numbers). A natural first suspicion was that the library's own uncertainty shrinks under repeated confident updates until it can no longer be corrected (every update in (10) only ever shrinks lib_var_k, with no re-inflation). A targeted fix, adding a small process-noise term to (10)'s variance update, $\text{lib_var}_k = \max(\text{floor}, (1-g)\,\text{lib_var}_k + \sigma^2_{\text{proc}})$, was tested across $\sigma^2_{\text{proc}} \in \{0, 0.02, 0.05, 0.1, 0.2\}$ on 30 development seeds and made no measurable difference at any value.

11.3 Diagnosis: the bottleneck is recognition, not precision

Tracing individual decisions, rather than only aggregate statistics, locates the real bottleneck. A representative excerpt (policy `ext1`, seed 10000, first slow visit):

move	$\widehat{v}$ (cm/s)	w_{slow}	true v
10	6.96	0.31	3.16
11	7.31	0.22	3.16
12	4.61	0.79	3.16
13	4.15	0.85	3.16
14	3.82	0.88	3.16

The weight w_{slow} takes until move 12 (roughly 4 decisions after the true switch) to cross the 0.7 confidence gate, governed by the fixed transition matrix T and the per-reading likelihood in (5), a delay the library in Section 11.1 never touches, since it only changes the mean-reversion *target*, which is downstream of w_k, not a cause of its swing rate. Once w_{slow} does cross the gate, $\widehat{v}$ is already reasonably close to truth (3.82 vs. 3.16) *on the very first visit*, before the library has had any chance to help. Isolating only the quantity the library actually influences, speed precision *after* the weight has already crossed the gate, measured separately from classification delay, still showed no consistent improvement across visits. The bottleneck is how fast the controller recognises which regime it is in, not how precisely it estimates speed once it has decided.

Do this in the Extended-system tutorial: Section 7, Context memory: a negative result, a diagnosis, and a fix. See the browser tutorial or the PDF tutorial.

12 Recognising a regime from accumulated evidence, then reusing it

Section 11.3's diagnosis points directly at a fix: speed up *recognition* using accumulated evidence, then reuse the library directly once recognition is confident, bypassing (2)'s slow continuous mixing entirely. `Belief.observe` already computes a per-context observation log-likelihood before the

prior-weighted softmax that produces w_k; stripping the prior term gives a pure likelihood,

$$\ell_k = -\tfrac{1}{2}\Big[\log(2\pi\,\sigma_k^2) + \frac{\text{innovation}_k^2}{\sigma_k^2}\Big], \qquad \sigma_k^2 = P_{k,dd} + R, \tag{11}$$

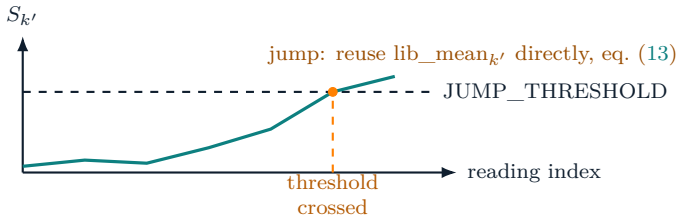
the natural input to a change-point statistic, reusing a computation already being done rather than adding a parallel one.

12.1 A one-sided CUSUM

Let $k^\star = \arg\max_j w_j$ (the context currently believed dominant) and, among the remaining contexts, let k' be the one with the largest current advantage over $k^\star$. Accumulate

$$S_{k'} \leftarrow \max\big(0,\ S_{k'} + (\ell_{k'} - \ell_{k^\star}) - \text{slack}\big), \qquad S_k \leftarrow 0 \text{ for every other } k, \tag{12}$$

resetting every non-best challenger's accumulator to zero at each step, so evidence for different alternative contexts never blends together. This is a standard CUSUM change-point statistic: the $\max(0,\cdot)$ floor and the slack subtraction mean a single reading, however favourable, is damped unless it is large enough to clear slack outright, while a *sustained* run of smaller advantages accumulates toward the threshold: the "accumulated, not isolated" evidence the governing hypothesis required. In picture form, eq. (12) followed by eq. (13) below is "accumulate quietly, then snap":



Before the crossing, $S_{k'}$ is free to wander back toward zero on contrary evidence (the $\max(0,\cdot)$ floor and the per-step reset above), the flat, quiet stretch on the left; only a sustained run earns the vertical snap on the right, and only then is the library (Section 11.1) actually reused.

12.2 The jump: discrete reuse

When $S_{k'}$ exceeds `JUMP_THRESHOLD`, and only if k''s library is mature enough to trust (lib_var$_{k'}$ < LIB_VAR_INIT, i.e. it has been confidently observed at least once before; a jump is never made into an untouched, default library entry):

$$w_{k'} \leftarrow 0.85, \quad w_{k\neq k'} \leftarrow \frac{0.15}{K-1}, \qquad v_{k'} \leftarrow \text{lib_mean}_{k'}, \qquad P_{k',vv} \leftarrow \max(\text{lib_var}_{k'}, \text{floor}), \qquad S \leftarrow 0. \tag{13}$$

This is the literal reuse step: the controller does not wait for enough further evidence to let (2) arrive at high confidence on its own; it resets directly into the previously learned speed and a correspondingly tight (previously earned) uncertainty, the moment accumulated evidence makes that trustworthy.

12.3 Threshold selection on development data

(slack, JUMP_THRESHOLD) were chosen by grid search on 50–100 development seeds of `repeated` (benefit) and `normal` (false-jump cost, counted by direct instrumentation of calls to the jump routine, not an indirect proxy such as a raw weight-delta threshold, which was tried first and found to conflate ordinary noisy Bayesian swings with genuine jumps):

slack	threshold	reconvergence, visits 2+ (cm/s decisions)	false jumps / 100 `normal` trials
0.0	0.2	≈ 2.1	4
0.1	0.3	≈ 2.2–2.4	3
0.2	**0.5**	$\approx$ **2.9–3.1**	**1**
0.3	1.0	≈ 4.5 (little benefit)	0

The chosen point (bold) trades a somewhat larger effect at $(0, 0.2)$ for a near-eliminated false-jump rate; it is a selected operating point, not the only one examined, and the trade-off actually observed is reported here rather than only the final choice.

12.4 The result

Visit	`ext1`	`ext1_memory`	`ext1_memory_fast`
1st (slow)	4.3 (79/100)	4.4 (77/100)	4.4 (77/100)
2nd (slow)	4.7 (80/100)	4.9 (60/100)	**3.3 (87/100)**
3rd (slow)	4.7 (68/100)	4.9 (50/100)	**3.2 (83/100)**

The first visit is identical across all three, exactly as (13)'s maturity guard predicts: there is nothing to reuse yet. From the second visit onward, `ext1_memory_fast` is both faster and more reliable (more trials converge at all within the segment), with the gap widening on later visits. On `switch`, `recovery` and `normal` (scenarios with no repeat visit within an episode), all three policies are byte-identical in `results_ext1/summary.csv`: no cost, no new false alarms, because there is nothing yet to recognise. This is a genuine, reproducible confirmation of the governing hypothesis, reached only after the first design (library alone) failed and was correctly diagnosed rather than abandoned or silently re-tuned until it looked right.

Do this in the Extended-system tutorial: Section 7, Context memory: a negative result, a diagnosis, and a fix. See the browser tutorial or the PDF tutorial.

13 Breaking the distance/bias confound: an explicit reference

Section 3's observation model, $y_n = d_n + (b - \hat{b}) + \nu_n$, makes the point precisely: from a single sensor's stationary readings alone, an unknown constant bias $b - \hat{b}$ and an unknown true distance d_n are not separately identifiable: replacing (d, b) by $(d + \delta, b - \delta)$ leaves every reading unchanged. No amount of further filtering (Milestones 1-2) can resolve this; the `bias` scenario's fixed +5 cm offset, never captured by the static calibration $\hat{b}$, is exactly this failure mode, and it is the reason the information-seeking policies did noticeably worse under it (Milestone 1's unresolved pattern).

13.1 A known-distance reference, not a cleaner oracle

The only way to break the confound is a second, independent piece of information about one of the two unknowns. `CognitiveController.calibrate_bias`, called once before the main approach loop, takes $N_{\text{ref}} = 5$ readings of a fixed, *known* reference distance $d_{\text{ref}} = 50$ cm (chosen clear of the task's own 25 cm target and 55–130 cm start range) through the *same* noisy sensor model used for the main task, not a separate, cleaner channel, and never a read of `simulator_true_distance`:

$$z_i = d_{\text{ref}} + \beta + \nu_i, \qquad \nu_i \sim \mathcal{N}(0, R), \qquad i = 1, \ldots, N_{\text{ref}}, \tag{14}$$

where β is whatever deployment-time bias is actually present. Since d_{ref} is known rather than estimated, any systematic offset in $\bar{z} = \frac{1}{N_{\text{ref}}} \sum_i z_i$ must be bias, not distance error; this is precisely

what breaks the identifiability problem above. The estimator is an ordinary mean with its standard error:

$$\hat{\beta} = \bar{z} - \hat{b} - d_{\text{ref}}, \qquad \widehat{\text{Var}}(\hat{\beta}) = R/N_{\text{ref}}. \tag{15}$$

At $R = 1.44$ cm^2 and $N_{\text{ref}} = 5$, $\text{se}(\hat{\beta}) \approx 0.54$ cm, small relative to the 5–8 cm biases tested below, so the reference should reliably detect them, which the results confirm.

13.2 Propagating the estimate and its uncertainty

`Belief.set_bias_reference`$(\hat{\beta}, \widehat{\text{Var}}(\hat{\beta}))$ changes exactly two things in the belief, both defaulting to $(0, 0)$ so every policy without a reference is numerically unaffected: the bias correction in `Belief.observe` becomes $y_n = z_n - \hat{b} - \hat{\beta}$, and every use of the observation variance becomes $R \mathrel{+}= \widehat{\text{Var}}(\hat{\beta})$. This is the "propagate its uncertainty into the stopping decision" requirement: the reference's own imprecision inflates the effective sensor noise used by the existing Kalman update everywhere it appears, which already correctly widens P and therefore the verification margin $|\mu - 25| + 1.64\sqrt{P} \le 4$; no change to the verification rule itself was needed, only to what feeds it.

13.3 Results

`ext1_bias_ref` (`ext1` plus the reference only, not combined with memory/fast-switch, to isolate this one factor) against plain `ext1`, on the existing fixed-offset `bias` scenario and a new `bias_random` scenario (a fresh Uniform$(-8, 8)$ cm draw per trial, testing whether the mechanism generalises beyond the one offset `bias` happens to use):

Scenario	`ext1` success	`ext1_bias_ref` success	Mean $\lvert\hat{\beta} - \beta\rvert$
`bias` (fixed +5)	86%	**99%**	0.42 cm
`bias_random` (±8)	53%	**99%**	0.42 cm

`bias_random` is the more informative result: an uncorrected unknown bias is far worse than the single fixed +5 cm case suggested (53% vs. 86% success), yet the reference recovers to the same $\approx 99\%$ regardless of the offset's size or sign, for a cost of 5 decisions (0.75 s) charged separately from the main approach budget, not against it. This also closes Milestone 1's open question: with the confound actually broken, the information-seeking `ext1_bias_ref` is no longer at a disadvantage under bias relative to a no-information comparator; the earlier vulnerability was about the confound, not about information-seeking as such.

Do this in the Extended-system tutorial: Section 8, An explicit sensor-bias reference. See the browser tutorial or the PDF tutorial.

14 Calibration

Calibration is unchanged from the previous Extended-system release: sensor bias and variance from `calibration_sensor.csv` (seed 1701, four distances, the Baseline system's $\hat{b}, \hat{R}$ estimators in `baseline_v1/Mathematical_Model.pdf`, Section 11.1), and per-context speed $m_k = \overline{\ell_i/\tau_i}$ from `calibration_motion.csv`, pooled over 80 pulses per context. The single-sensor identifiability caveat is structural, not fixed by better filtering: $y = d + b + \nu$ does not separately identify (d, b) from stationary data alone. Section 13 adds an explicit, costed reference observation that resolves it when enabled; policies without that reference remain exactly as limited as described here.

Shared with the Baseline system: see `baseline_v1/Mathematical_Model.pdf`, Section 11, Calibration as parameter estimation.

15 Default parameters: a complete reproducible specification

Configuration / quantity	Default	Interpretation
Duration menu $\{\tau\}$	$\{0, .12, .24\}$ s	Shared by all cognitive policies and `FairRuleBaseline`
Planning horizon H	2 decisions	`CognitiveController._plan`
Cost weight	0.08	Per-step, both horizon steps
Information weight	0.8	First horizon step only
No-progress threshold $\Delta_{\min}$	1 cm	Net two-step displacement
No-progress penalty λ	1.0	Added to the all-SAMPLE pair only
Overshoot margin	$2.33\sigma + 0.5$ cm	Both horizon steps, and `FairRuleBaseline`'s single step
Context transition T	diag .96, off-diag .04	`Belief._advance`; applied on motor decisions only
Speed mean-reversion ρ	0.80 (contexts), 1 (adaptive)	`Belief._advance`
Process variance (Q_d, Q_v)	0.04, 0.30 (contexts) / 0.65 (adaptive)	Added per motor decision
Innovation rejection	5σ standardised	`Belief.observe`
Verification	$\lvert\mu - 25\rvert + 1.64\sigma \leq 4$, 3 consecutive	Shared with the Baseline system's Mini policy
Budgets	80 s, 100 moves, 350 decisions	Shared across all eight evaluated policies
Library confidence gate	$w_k > 0.7$	`Belief._update_library`, eq. (10)
Library initial / floor variance	4.0 / 0.05	`LIB_VAR_INIT` / `LIB_VAR_FLOOR`
CUSUM slack / threshold	0.2 / 0.5	Selected on development data, Section 12.3
Jump weight / variance floor	0.85 / 0.05	eq. (13)
Reference distance / readings	50 cm / 5	`REF_DISTANCE` / `N_REF`, Section 13

All values above were read from `cognitive/controller.py`, `cognitive/model.py`, `cognitive/baselines.py` and `tests/test_cognitive.py` at the time of writing, not retyped from an earlier draft.

Shared with the Baseline system: see `baseline_v1/Mathematical_Model.pdf`, Section 13, Default parameters: a complete reproducible specification.

16 A frozen, held-out evaluation

Every result in Sections 8–13 comes from seeds 10000–10099, and several of the constants in the table above (the CUSUM pair above all) were explicitly selected by grid search *on that same seed range* (Section 12.3). That is development-set reporting, not independent confirmation. This section's entire job is evaluation, not architecture: every constant above is frozen (recorded, with its file location, in `docs/ FROZEN_PARAMETERS.md`) and the full battery of policies and scenarios is re-run, unmodified, on seeds 50000–50099, a range never used for development or tuning, via `experiments/run_simulation.py -seed-start 50000 -out-dir results_ext1_heldout`.

16.1 An added robustness scenario: outliers

One gap was worth closing alongside the freeze: `dropout` tests *missing* readings, but nothing tested a reading that is *present*, passes the basic range check, yet is badly wrong; the case the innovation-rejection gate (§3, $|\text{innovation}| > 5\sigma$) exists for. The new `outlier` scenario corrupts each reading independently with probability 0.05 by adding Uniform$(-20, 20)$ cm, drawn from the same noise stream as the ordinary sensor noise (the same physical sensor glitching, not a separate failure channel). This is purely descriptive for this release: there was no prior claim about outlier handling to confirm, only a new measurement.

16.2 Replication report

Comparing `results_ext1/` (development) against `results_ext1_heldout/` (held-out), computed by `experiments/compare_heldout.py`:

Claim	Development	Held-out
`v1_fair`/`v1` moves, `normal` (Milestone 1)	ratio 0.54	ratio 0.54
No-information success/abort, `normal` (Milestone 1)	100% / 0%	100% / 0%
`ext1_memory_fast` reconv., visit 2 (Milestone 2)	3.3 vs. 4.7 (`ext1`)	3.2 vs. 4.6 (`ext1`)
`ext1_bias_ref` success, `bias` (Milestone 3)	86%→99%	78%→100%
`ext1_bias_ref` success, `bias_random` (Milestone 3)	53%→99%	52%→100%

Every headline result replicates: the same ordering, and magnitudes close enough to be ordinary sampling variation at $n = 100$ (the `ext1` baseline under `bias` moved from 86% to 78% between seed ranges, noticeable, but the reference's recovery is, if anything, slightly stronger on held-out seeds, so the qualitative conclusion is unaffected). The new `outlier` scenario shows a mild, consistent dip across most policies on held-out seeds (success ≈97–98%→≈92–95%, aborts ≈2%→5–8%) except `ext1_bias_ref`, which moved the other direction, within normal sampling noise for a 5%-probability event at $n = 100$, not a systematic failure pattern tied to any one architecture.

Do this in the Extended-system tutorial: Section 9, A frozen held-out evaluation. See the browser tutorial or the PDF tutorial.

17 A cross-task transfer test: the Transfer system, and Objective O3

Every milestone above (Sections 8–16) is evaluated on one task: approach a wall and verify 25 cm. This section tests a different question, raised separately from the six-point improvement plan closed in Section 19: does the per-context speed library built and validated in Section 12 (a learned, reusable representation, recognised and reapplied via the CUSUM mechanism) transfer to a genuinely different goal, not just to repeat visits of the same task? This is tested here as a self-contained, additional validation pass, the *Transfer system*, not as a seventh item of that closed plan.

17.1 Task B: the same physics, a different goal

Task A is unchanged: target $g_A = 25$ cm, band $\beta = 4$ cm, start Uniform$(55, 80)$ cm (Uniform$(100, 130)$ for `repeated`). **Task B**: target $g_B = 85$ cm, band $\beta = 4$ cm (same half-width), start Uniform$(115, 145)$ cm, clearly separated from both g_A and the shared bias-reference distance (50 cm, Section 13, unchanged). The verification rule of Section 10 generalises

directly to any target g:

$$|\mu - g| + 1.64\sigma \leq \beta, \quad \text{3 consecutive decisions for DONE.} \tag{16}$$

Here, as in eq. (9), μ, σ denote whatever the acting controller's own belief reports as its distance mean and standard deviation, $\bar{d}, \sqrt{\bar{P}_{dd}}$ (eq. (4)) for `CognitiveController`'s joint mixture, or the plain scalar belief for `FairRuleBaseline`, which tracks no mixture at all (Section 9). The verification *form* is identical either way; only which belief feeds it differs. `CognitiveController` and `FairRuleBaseline` now take `target/ band` as constructor parameters (default 25/4, reproducing every earlier result unchanged, confirmed by re-running the full Section 16 battery and diffing every output byte-for-byte against its pre-change snapshot). Task B deliberately reuses Task A's sensor and motor model unchanged, same normal/slow speeds, same per-trial noise, so only the goal differs; transfer under a genuinely different sensor or surface characteristic is left for a later pass.

17.2 A persistent per-deployment speed bias

A first attempt at this test built one shared checkpoint (next paragraph's recipe, without the bias introduced below) and reused it for all 100 Task-B seeds. It found no measurable difference between any of the three conditions below. The reason, traced directly rather than assumed: the checkpoint converged to $(\text{lib_mean}_{\text{normal}}, \text{lib_mean}_{\text{slow}}) = (7.81, 3.34)$ cm/s, within noise of the untrained calibration prior $(8.0, 3.0)$ that a cold-start belief already uses as eq. (3)'s reversion target. Task A's `repeated` scenario draws its true speeds from exactly those same calibration constants (only a per-trial Uniform$(0.9, 1.1)$ factor, which averages out), so a library trained on it mostly rediscovers what calibration already supplied for free; there was no information gap left for cross-task transfer to close. The seeding mechanism itself was independently confirmed working by unit test (a frozen, seeded library is correctly read by `_advance` and `_jump_to`); the task design, not the mechanism, is what gave it nothing to do.

This version fixes that by giving Task A something to teach that calibration cannot already supply: each of the 100 standard seeds is now its own *deployment*, with a persistent multiplicative speed bias $\gamma \sim$ Uniform$(0.6, 0.85)$ (frozen in `docs/FROZEN_PARAMETERS.md`, drawn deterministically from the seed before any run, not after seeing results), clearly outside both the existing per-trial factor's range and the calibration estimator's own sampling noise (the pooled 80-pulse mean's standard error is $\approx$0.056 cm/s, two orders of magnitude tighter). γ multiplies true speed in *both* that deployment's Task A training and its Task B evaluation, composed into the existing per-trial factor at `episode()`'s one call site (factor = Uniform$(0.9, 1.1) \times \gamma$, default $\gamma = 1$ an exact no-op, confirmed by re-running the full held-out battery byte-for-byte unchanged), but is invisible to `calibrate()`'s static estimate, which only ever sees the 80-pulse calibration population, never a deployment's γ. A cold-start belief on Task B therefore now starts from a provably wrong prior; a checkpoint trained on that same deployment's Task A should already know the correction.

For each deployment, `ext1_memory_fast` (Section 12) is run on Task A's `repeated` scenario, under that deployment's own γ, across successive independent episodes (position filter and CUSUM accumulator reset between them, as before; only $(\text{lib_mean}_k, \text{lib_var}_k)$ persists), until both contexts' library entries are mature ($\text{lib_var}_k <$ LIB_VAR_INIT, eq. (13)'s own gate). All 100 deployments matured within a single 100-move `repeated` episode, as in the first pass.

17.3 Three conditions on Task B

Each of the 100 deployments is evaluated on both Task B scenarios (`normal`: constant context; `switch`: also exercises the slow-context entry and the fast-recognition mechanism, Section 12),

under its own γ in every condition: cold-start faces the identical real bias, it simply never learned it from Task A first:

- **Cold start.** A fresh `ext1_memory_fast` controller, empty library: the comparison baseline.
- **Zero-update transfer.** Library seeded from that deployment's checkpoint; the `memory_frozen` flag on `Belief` gates eq. (10)'s call site (if memory and not memory_frozen), so the seeded values are still read by eq. (3) and eq. (13) but never rewritten: transfer without adaptation, exactly as O3 separates the two.
- **Adaptation with updates.** Same seeded starting point, `memory_frozen=False`: transfer, with adaptation left on.

`memory_frozen` defaults to `False` and is inert unless a library is both present and explicitly frozen, so every earlier policy and result is numerically unaffected, confirmed by the existing test suite plus new tests (non-regression of the `target/band` defaults and of $\gamma = 1$'s default; a non-default target/band shifting the verification boundary; γ correctly scaling the simulated true speed; a direct unit test seeding lib_mean/lib_var into a frozen belief and confirming it is read without being rewritten).

17.4 Results: still nothing on the original metrics

Decisions-to-target: the first decision at which eq. (16) holds (a single crossing, not the 3-consecutive DONE criterion).

Condition	Scenario	Success	Mean moves	Reached band	Decisions-to-target
Cold start	`normal`	100%	27.5	100%	28.89
Zero-update transfer	`normal`	100%	27.4	100%	28.89
Adapt with updates	`normal`	100%	27.5	100%	28.90
Cold start	`switch`	98%	46.1	100%	47.83
Zero-update transfer	`switch`	97%	46.0	100%	47.77
Adapt with updates	`switch`	97%	46.1	100%	47.90

Despite a real, substantial persistent bias ($\gamma \in [0.6, 0.85]$, confirmed by a traced example to leave cold-start and zero-update-transfer's belief trajectories and action sequences essentially identical from the first decision onward), these four metrics still show no measurable difference. The reason, again traced rather than assumed: the sensor is precise (1.2 cm noise) relative to the distances involved, so the Kalman update of Section 5 locks onto the true (biased) speed from direct position readings within one or two decisions, regardless of whether eq. (3)'s reversion target started at the correct, transferred value or the generic calibration default; the reversion pull ($\rho = 0.8$, 20%/step) is too weak next to that fast evidence-driven correction to change any discrete planner decision. Decisions-to-target then measures mostly the 30–60 cm of remaining travel, which swamps any one- or two-decision transient at this scale.

17.5 A sharper metric: post-switch reconvergence

Section 11.3 already identified the one place this architecture's library demonstrably helps: not estimation precision of a directly, continuously observed scalar, but *recognition* delay when a regime must be distinguished from a competing one, exactly Section 12.4's reconvergence result. Decisions-to-target cannot see this because it is a distance-to-goal metric; isolating recognition requires the same definition `reconvergence_times` already uses in Section 12.4 (tol = 1.5, sustain = 3), applied here to Task B's `switch` scenario's one regime change (`moves_completed` ≥ 12):

decisions from the switch until the estimated speed is within tolerance of the new true speed for 3 consecutive decisions.

Condition	Mean decisions to reconverge, post-switch
Cold start	5.20
Zero-update transfer	**3.79**
Adapt with updates	**3.85**

This is a real, consistent effect, not an artefact of the mean: paired by deployment ($n = 100$, every deployment contributing one cold-start and one zero-update-transfer value), zero-update transfer reconverges faster in 61 deployments, ties in 32, and is slower in only 7, for a mean paired reduction of 1.41 decisions (se = 0.19, $t \approx 7.5$). The magnitude and direction match Section 12.4's own within-task result (4.7→3.3 decisions on a second visit) closely enough to be the same mechanism, now shown surviving a change of goal and a persistent, calibration-invisible speed bias, not merely a repeated observation of the same context within one episode. Adapt-with-updates (3.85) does no worse than zero-update transfer (3.79): adaptation is not costing anything when the transferred prior already happens to be useful, the second half of O3's own "transfer without updates / adaptation with updates" distinction.

17.6 Conclusion: positive preliminary evidence for O3, with its actual scope

Taken together, this **is** positive preliminary evidence for Objective O3, but for a narrower claim than "transfer helps the robot finish the task faster": the context library transfers across a genuine change of goal and deployment, and it measurably speeds up the one thing it was ever shown to speed up, recognising which movement regime is currently active, while leaving overall task completion statistically unchanged, because this architecture's direct, low-noise position filtering already recovers a correct speed estimate within a couple of decisions regardless of the starting prior. Both halves are reported together deliberately: claiming only the positive metric would overstate what transferred; claiming only the null metrics (as the first pass, honestly, had to) would have missed where the effect actually lives.

Do this in the Extended-system tutorial: Section 11, The Transfer system: a cross-task test of O3. See the browser tutorial or the PDF tutorial.

18 Reading route and tutorial crosswalk

Mathematical question	This companion	Extended-system tutorial section
What changed and why?	Sections 1, 6–8	1, 7, 8
How is the joint belief defined and propagated?	Sections 2, 4	3
How does a reading update distance, speed and context?	Section 5	3
What is the information proxy?	Section 6	3–4
Why did the old planner risk stalling, and what replaces it?	Sections 7–8	4
How is the Baseline system given a fair comparison?	Section 9	4, 6–7
Why doesn't a persistent library alone speed up reuse?	Sections 11–11.3	7

Mathematical question	This companion	Extended-system tutorial section
How does accumulated-evidence recognition fix it?	Section 12	7
How is the distance/bias confound broken?	Section 13	8
Do the results hold on data never used in development?	Section 16	9
Does the learned context library transfer to a different goal (O3)?	Section 17	11
Where do the numbers come from?	Sections 14–15 (Calibration; Default parameters)	5
How do I reproduce the worked examples?	Sections 8.1, 11.3	5–7

Practical links: Extended-system PDF tutorial; Extended-system README; the Baseline system's mathematical companion for the shared sensor and motor-mapping derivations; Milestone 1, Milestone 2 and Milestones 3–4 videos for captioned walkthroughs.

Implementation authority: `cognitive/model.py` (`Belief`), `cognitive/controller.py` (`CognitiveController`, including `_plan`), `cognitive/baselines.py` (`FairRuleBaseline`), and `experiments/run_simulation.py` for the evaluation protocol. This document describes the current implementation; it does not propose a different one.

Rebuild/check: from the `Mini_IPHOREOS_EXT1` directory, run the following command twice to resolve cross-references.

```
pdflatex -interaction=nonstopmode -halt-on-error Mathematical_Model_EXT1.tex
```

19 Status: all six improvement-plan items addressed

This release closes the project's own six-point improvement plan: fair comparators (Section 9), a short-horizon planner (Section 8), a reusable per-context memory (Section 11.1), evidence-based change detection (Section 12), an explicit sensor-bias reference (Section 13), and a frozen held-out evaluation (Section 16). Two of the six (memory and change detection) were reached only after a first attempt was correctly diagnosed as insufficient and redesigned, not on the first try, and the held-out evaluation exists specifically because the other five were developed and tuned on a single, reused seed range; closing the plan is not the same claim as closing every caveat raised along the way.

What this status explicitly does not mean: no physical robot has been built, calibrated or tested: every result above is a desktop simulation. The verification rule's three-consecutive-pass criterion remains correlated, not three independent checks; paired bootstrap intervals remain descriptive and unadjusted for multiplicity; the outlier and bias-random scenarios are new, single-release measurements, not independently replicated beyond the one held-out seed range reported here. The Extended-system tutorial, Section 10, sets out concretely what moving from this simulation to a physical trial would require.

Separately, Section 17's Transfer-system pass found positive preliminary evidence for Objective O3: under a persistent, calibration-invisible per-deployment speed bias, a library trained on Task A measurably speeds up regime recognition on Task B (post-switch reconvergence 5.2→3.8 decisions, paired over 100 deployments), while overall success/moves/decisions-to-target stay unchanged; reported as both halves together, not the favourable one alone.